



INGENIERÍA EN SISTEMAS DE INFORMACIÓN
Quinto año

CÁTEDRA:
Proyecto Final - Ingeniería en Sistemas de Información

Ciclo de Vida y Enfoque de desarrollo

DOCENTES:

Ing. Christian Villafañe
Ing. Matías Cassani

EQUIPO: Five stack

ALUMNOS:

Liendo, Alejo.	Legajo N° 15074	alejoliendo2004@gmail.com
Lucarelli, Bruno.	Legajo N° 14988	brunolucarelli5@gmail.com
Magni, Gastón.	Legajo N° 14991	gastonmagni@hotmail.com
Mosconi, Ignacio.	Legajo N° 15288	ignamosconi@gmail.com
Terreno, Valentino.	Legajo N° 15079	ninot2016@gmail.com

ÍNDICE

ÍNDICE	2
HISTORIAL DE REVISIONES	4
CICLO DE VIDA DEL PROYECTO	5
Justificación de la selección	5
Las 5 fases del ciclo de vida del proyecto	5
Representación del ciclo de vida	7
ENFOQUE DE DESARROLLO DEL PRODUCTO - COMPONENTE ADAPTATIVA	7
Introducción	7
El PUD dentro de cada sprint	7
1 - Workflow de requisitos	8
2 - Workflow de análisis	8
3 - Workflow de Diseño	8
4 - Workflow de implementación	9
5 - Workflow de prueba	9
Planificación de los Sprints	9
1 - Definition of Done	9
2 - Historias de usuario	10
2.1 - Ejemplo de User Stories:	11
2.2 - Criterios de aceptación generales:	12
2.3 - Flujo de una historia hasta Testnet:	13
3 - Ceremonias	13
3.1 - Sprints:	13
3.2 - Reuniones:	13
3.3 - Sprint planning:	14
3.4 - Sprint review:	14
3.5 - Sprint retrospective:	14
4 - Artefactos	14
4.1 - Backlogs y artefactos	14
4.2 - Minutas	15
4.3 - Métricas:	16
5 - Estructura del equipo y Matriz RASCI	17
5.1 - Asignación de roles por fase - Matriz RASCI:	17
5.2 - Requerimientos:	18
5.3 - Roles en Scrum:	18
6 - Flujo de Scrum simplificado	18
7 - El rol de Jira en nuestros Sprints	19
ENFOQUE DE DESARROLLO DEL PRODUCTO - COMPONENTE PREDICTIVA	22
Introducción	22
Herramientas del proceso	22
Plan de Gestión de la Configuración	22
Gestión de la configuración (SCM) del proyecto	23
Versionado documental	23

Procedimiento formal de cambios	23
Roles e integrantes del comité	24
Procesos de auditoría	24
Gestión de la configuración del producto	24
Sistema de control de versiones (Software)	24
Convención para Commits	24
Gestión de cambios al alcance	25
Gestión de entornos	26
Gestión de artefactos de release	26
Roles e integrantes del comité	26
Procesos de auditoría	27
BIBLIOGRAFÍA	28

HISTORIAL DE REVISIONES

Versión	Fecha	Autor/es	Descripción
1.0.0	21/04/26	Bruno Lucarelli	Creación inicial del documento. Planteo de títulos a completar. Formato general del documento.
1.1.0	25/04/26	Valentino Terreno	Redacción inicial de los títulos: ciclo de vida del proyecto, Enfoque de desarrollo, Enunciado del alcance (Scope statement), Estructura de desglose del trabajo (EDT / WBS), Diccionario de la EDT, Registro de supuestos y restricciones, Gestión de la configuración.
1.2.0	25/04/26	Gastón Magni	Redacción de Métricas y matriz RASCI.
1.2.1	25/04/26	Bruno Lucarelli	Formato, revisión documento, resolución comentarios.
1.2.2	26/04/26	Alejo Liendo	Revisión de contenidos
1.2.3	26/04/26	Gastón Magni Bruno Lucarelli Ignacio Mosconi Valentino Terreno	Revisión de contenidos, formateo general y resolución de dudas pendientes.
1.2.4	26/04/26	Ignacio Mosconi	Refactoring del título "Componente adaptativa - Scrum simplificado". Inclusión de imágenes en título "Criterios de finalización (Definition of Done)".
1.2.3	27/04/26	Valentino Terreno	Corrección de formateo de tablas y bullet points.
1.3.1	03/05/26	Ignacio Mosconi	Refactor del título "Enfoque de desarrollo del producto".
1.4.0	03/05/26	Bruno Lucarelli	Workflow de Requisitos, Workflow de Análisis, Workflow de Diseño, Workflow de Implementación, Workflow de Prueba, Ceremonias, Artefactos, revisión de contenidos y corrección formato.
1.5.0	04/05/26	Valentino Terreno	Formateo de alineación de párrafos. Refactoring del título Gestión de la configuración, separación entre proyecto y producto.
1.5.1	04/05/26	Gastón Magni Ignacio Mosconi	Aplicación de correcciones realizadas por la cátedra vía correo electrónico el día 04/05/26
1.5.2	23/05/26	Ignacio Mosconi	Mención de DER en WF de Diseño.

CICLO DE VIDA DEL PROYECTO

El ciclo de vida de VOTAR es de tipo híbrido, combinando una fase inicial predictiva con fases de desarrollo iterativas e incrementales. Esta elección se justifica por las características particulares del proyecto: por un lado, los requerimientos de gestión de proyectos y documentación académica son conocidos y estables (favoreciendo lo predictivo); por otro, la implementación del stack blockchain-backend-frontend implica complejidad técnica, innovación y necesidad de retroalimentación continua (favoreciendo lo adaptativo).

Justificación de la selección

Para fundamentar esta elección se analizan los factores del *PMBOK® 7ma edición* que influyen en la selección del ciclo de vida:

Factor	Análisis en VOTAR	Implicancia
Grado de innovación	Alto: blockchain + ECC + Merkle son tecnologías novedosas para el equipo	Enfoque adaptativo en desarrollo
Certidumbre en requisitos	Alta para gestión y documentación; media-baja para componentes técnicos	Híbrido: predictivo para PM, iterativo para software
Estabilidad del alcance	Estable a nivel funcional; variable a nivel de implementación técnica	Incrementos para el producto de software
Requisitos de seguridad	Muy elevados: criptografía, inmutabilidad, anonimato	Planificación rigurosa al inicio; validación iterativa
Tamaño del equipo	5 personas (dentro del rango 7 ± 2 de enfoques ágiles)	Facilita métodos adaptativos
Restricciones de cronograma	Plazos académicos fijos.	Predictibilidad necesaria para cumplir hitos

Las 5 fases del ciclo de vida del proyecto

El ciclo de vida del proyecto se estructura en cinco fases principales. La cadencia de entrega es múltiple: cada fase produce entregables específicos, y la fase de construcción genera incrementos de software funcional en cada sprint.

Si bien la tabla que se presenta a continuación cataloga a cada fase dentro de un rango de sprints, cabe destacar que dicho rango corresponde al **inicio** de cada fase, lo que implica que las fases pueden desarrollarse en paralelo (por lo que no es necesario que una finalice para dar comienzo a la siguiente, recordemos que estamos utilizando un enfoque ágil). En este sentido,

por ejemplo, la fase 2 puede iniciar aproximadamente en el sprint 6, aunque esto puede adelantarse o retrasarse, permitiendo así la ejecución simultánea de distintas fases.

Fase	Nombre	Actividades principales	Entregable clave	Sprint de inicio
1	Inicio del proyecto	Definición de la visión del producto y propósito del proyecto; identificación de interesados principales; definición de alto nivel del alcance; conformación del equipo, clarificación de roles y adopción de Scrum como marco de trabajo	Documentación inicial aprobada (Project Charter, registro de interesados, alcance de alto nivel)	1–5
2	Preparación y planificación inicial del producto	Identificación y priorización de requisitos funcionales en el backlog inicial; definición de arquitectura técnica preliminar (smart contracts, backend API, árbol de Merkle, diseño de BUD); establecimiento de acuerdos de trabajo, criterios de calidad, definición de terminado y hoja de ruta de entregas	Backlog inicial priorizado, arquitectura preliminar validada y plan de entregas	6–8
3	Desarrollo iterativo e incremental	Ciclos de sprint: análisis, diseño, desarrollo, prueba e integración de incrementos funcionales; implementación de autenticación SSO, emisión de voto, recibo criptográfico y Dashboard público; revisión con interesados y actualización de prioridades al cierre de cada sprint	MVP con flujo E2E funcional	9–12
4	Validación y liberación del producto	Consolidación de pruebas finales y validación funcional; proceso electoral simulado (CEUTI) como escenario de aceptación; auditoría de resultados;	Demo piloto auditada	13–14

Fase	Nombre	Actividades principales	Entregable clave	Sprint de inicio
		revisión de criterios de aceptación y ajustes previos al despliegue		
5	Cierre y transición	Formalización de la aceptación del producto; publicación open source del código; elaboración de documentación técnica y tomos finales; presentación ante evaluadores; recopilación de lecciones aprendidas	Tomos finales + presentación	15–17

Representación del ciclo de vida

Las fases 1 (Inicio/Planificación) y 5 (Cierre) operan bajo un **enfoque predominantemente predictivo**: los entregables están bien definidos, son reproducibles y el equipo puede planificar en detalle desde el inicio. Las fases 2, 3 y 4 operan bajo un **enfoque predominantemente adaptativo** con sprints de **dos semanas**, reuniones de sincronización y retrospectivas al finalizar cada incremento.

Este modelo corresponde al patrón “*desarrollo ágil seguido de despliegue predictivo*” descrito en el *PMBOK® 7ma edición*, adaptado al contexto académico del Proyecto Final de ISI.

ENFOQUE DE DESARROLLO DEL PRODUCTO - COMPONENTE ADAPTATIVA

Introducción

A continuación desarrollaremos el enfoque de desarrollo del producto, que nos permitirá mantener la trazabilidad y cumplimiento de los hitos académicos, al tiempo que gestiona de forma efectiva la incertidumbre técnica inherente al stack blockchain + criptografía aplicada.

Este título recibe el nombre de componente adaptativa ya que nuestro marco de trabajo base será una versión de **scrum simplificada** (*una forma ágil de trabajar*), adaptada justamente al tamaño del equipo y el contexto académico del proyecto. Cada sprint que trabajemos tendrá una duración de **DOS SEMANAS**, las cuales verán la realización de distintas ceremonias y la creación de diversos artefactos.

El PUD dentro de cada sprint

Dentro de cada sprint va a trabajarse la estructura del PUD (Proceso Unificado de Desarrollo), que nos indica qué actividades deben llevarse a cabo para poder gestionar un proyecto (*en este caso, de desarrollo de software*) de forma **iterativa** e **incremental**.

Los *workflows* del PUD que podemos trabajar dentro de un sprint son:

1 - Workflow de requisitos

El propósito fundamental del flujo de trabajo de los requisitos es guiar el desarrollo hacia el sistema correcto mediante una descripción y la correspondiente actualización de los requisitos del sistema, de tal manera que pueda llegarse a un acuerdo entre el cliente y los desarrolladores sobre qué debe y qué no debe hacer el sistema. Este flujo de trabajo incluye los siguientes pasos:

- Comprender el contexto del sistema.
- Capturar y actualizar requisitos funcionales y no funcionales.

En nuestro caso, los requisitos funcionales son capturados mediante **Historias de Usuario**. Estas Historias de Usuario serán documentadas y gestionadas utilizando la herramienta Jira, lo que nos permitirá mantener un **Product Backlog** ordenado y priorizado.

2 - Workflow de análisis

Durante el análisis, estructuramos y refinamos los requerimientos capturados previamente para lograr una mayor comprensión del sistema. En este workflow, utilizaremos una *vista simplificada del diagrama de clases* para modelar las entidades principales de la aplicación y sus interacciones fundamentales. Elegimos este enfoque porque presentar únicamente las clases y relaciones más relevantes facilita significativamente la comprensión y hace más manejable la exposición técnica por parte del equipo. Al evitar la complejidad inicial de un diagrama completo, establecemos una base clara y ágil que guiará la distribución del comportamiento y el desarrollo iterativo del proyecto. Para garantizar la integridad y el seguimiento del proyecto, todos estos artefactos de análisis serán almacenados en Google Drive y se versionarán en conjunto con el código fuente.

3 - Workflow de Diseño

Durante el diseño, transformamos los requisitos y las abstracciones del análisis en una especificación técnica detallada que servirá como guía fundamental para la implementación, definiendo la arquitectura que soportará con éxito tanto los requisitos funcionales como los no funcionales.

Para cubrir completamente el alcance del sistema, estructuraremos los componentes mediante dos niveles de modelado. A nivel arquitectónico, utilizaremos el modelo **C4** (Contexto, Contenedor y Componente), lo que nos permite documentar y comunicar la composición estática del software en diferentes niveles de abstracción de forma clara y escalable.

Complementariamente, a nivel de comportamiento, aplicaremos **diagramas de secuencia** UML para modelar el flujo dinámico de las operaciones críticas, tales como la autenticación SSO, la emisión del voto, la generación del recibo criptográfico y el escrutinio en tiempo real.

Además, realizaremos un **diagrama de entidad relación (DER)** que nos permita analizar la permanencia con la que vamos a trabajar en el sistema.

Elegimos esta combinación porque nos permite **cubrir completamente el alcance del sistema** y demostrar una sólida trazabilidad técnica a través de representaciones modulares, evitando la sobrecarga cognitiva al definir con precisión cómo interactúan el usuario, el frontend, el backend y las APIs involucradas.

Finalmente, para garantizar la integridad y el seguimiento del proyecto, todos estos artefactos de diseño serán almacenados en Google Drive y se versionarán en conjunto con el código fuente.

4 - Workflow de implementación

El workflow de implementación toma como punto de partida las especificaciones técnicas y arquitectónicas establecidas durante el diseño, con el objetivo de materializarlas en artefactos de software concretos. En esta etapa, el equipo se enfoca en la traducción de los modelos previamente concebidos en componentes reales y funcionales, abarcando la escritura del código fuente, la generación de scripts y la configuración de los entornos necesarios. El propósito central de este flujo es la construcción e integración paulatina del proyecto, consolidando las distintas piezas tecnológicas para ensamblar y desarrollar el sistema en su totalidad como un producto robusto.

5 - Workflow de prueba

En el flujo de trabajo de la prueba verificamos el resultado de la implementación probando cada construcción, incluyendo tanto construcciones internas como intermedias, así como las versiones finales del sistema a ser entregadas. Los objetivos principales de este workflow se pueden resumir de la siguiente manera:

- Planificar las pruebas necesarias en cada iteración, incluyendo las pruebas de integración, las pruebas de sistema y las unitarias.
- Diseñar e implementar las pruebas creando los casos de prueba que especifican qué probar.
- Realizar las diferentes pruebas y manejar los resultados de cada prueba simultáneamente.

Para manejar los resultados de las pruebas, se hará registro de feedback en las historias de usuario que correspondan y se actualizará el estado de la misma en el Jira del equipo. Así, el encargado de implementar la tarea puede corregir la funcionalidad. La historia de usuario será testeada hasta que los resultados sean satisfactorios y los tests sean pasados con éxito.

Planificación de los Sprints

1 - Definition of Done

Antes de desarrollar cómo vamos a implementar Scrum es pertinente entender el de la DoD o **Definition of Done**, que le permiten reconocer al equipo de trabajo cuándo una historia de usuario (es decir, un ítem del **product backlog**, algo que veremos más adelante) está terminada y lista para entregar.

La DoD cambia con cada equipo de trabajo, y en nuestro caso consideramos a un ítem como terminado cuando se cumple la totalidad de los siguientes criterios:

- El código fuente fue revisado y aprobado por al menos un compañero (pull request revisado y aprobado).
- Existen pruebas unitarias o de integración que cubren los casos de uso principales.
- El smart contract, si aplica, fue auditado localmente y desplegado en Testnet sin errores.
- La documentación técnica asociada (comentarios de código, README) fue actualizada.
- La documentación funcional asociada (*workflows del PUD, ciertos artefactos de scrum - como el product backlog, por ejemplo*) fue actualizada.
- No hay deuda técnica crítica pendiente relacionada con el ítem.
- El equipo realizará mediciones periódicas sobre dos dimensiones: el proceso de gestión y la calidad del producto. Las mismas serán tenidas en cuenta durante las Sprint Retrospectives, lo que permitirá ajustar el flujo de trabajo para futuros.

2 - Historias de usuario

El otro concepto a definir es cómo vamos a trabajar las historias de usuario de nuestro proyecto.

Una historia de usuario es una **explicación informal** de una funcionalidad de software, escrita desde la perspectiva del usuario final, que describe cómo esta funcionalidad le **aporta valor**. Estas siguen el siguiente formato:

Como [rol], quiero [acción], para [beneficio].

Las historias de usuario con las que vamos a trabajar incluye los siguientes campos obligatorios:

- a. ID: Identificador único brindado por Jira
- b. Epica: Módulo funcional al que pertenece
- c. Descripción: **como X, quiero Y, para Z**
- d. Criterios de aceptación: Condiciones concretas y verificables que debe cumplir la historia para considerarse completa
- e. Pruebas de usuario: Casos de prueba específicos derivados de los criterios, con entrada, acción esperada y resultado

- f. Prioridad: Alta / Media / Baja, asignada en el Sprint Planning
- g. Estado: Asignado / En proceso / En revisión / Terminado
- h. Story Points: Estimación del esfuerzo relativo del equipo. Para este caso utilizaremos las técnicas:
- Poker planning**: Cada integrante vota de forma simultánea; si hay divergencia, los extremos argumentan su postura y se revota hasta alcanzar consenso. El objetivo no es la precisión absoluta sino la consistencia relativa entre historias del mismo backlog.
 - Criterio Fibonacci**: El equipo asigna valores de la escala Fibonacci representando el esfuerzo estimado de la tarea.

Practical Fibonacci	
0	No effort is required , or there is some effort required, but there is no business value delivered, so no Points are accumulated for doing the work. An example is a desired behavioral change deriving from the Scrum Retrospective.
1	Extra small . Developers feel they understand most requirements and consider it relatively easy, probably the smallest item in the Sprint and mostly likely completed in one day.
2	Small . A little bit of thought, effort, or problem-solving is required, but the Developers have done this a lot, so they have confidence in the requirements. Or, it sounds extra small, but they want to hedge their bet just a bit.
3	Average . Developers have done this a lot; they know what needs to be done. There may be a few extra steps, but that's it. It's doubtful that they will need to research anything.
5	Large . This is complex work, or the Developers don't do this very often. Most Developers will need assistance from someone else on the team. This is probably one of the largest items that can be completed within a Sprint.
8	Extra Large . This is going to take some time and research and probably more than one developer to complete within two weeks. In addition, Developers need to make several assumptions that increase the risk and could affect getting it Done.
13	Warning! This is a complex piece of work with a lot of unknowns and requires multiple assumptions to size. It is too much to complete in one Sprint. Instead, split this into multiple items that can be completed independently.
21	Hazard! A "21" or "34" reflects too much complexity to be done within one Sprint. It will need to be refined more. The large size also indicates more risk, assumptions, and dependencies involved to complete this item.
?	Danger! As a Developer, we don't want to do this work the way it is currently written. It is very complex and cannot be completed in the timeframe of an iteration or Sprint. Perhaps the requirements are so fuzzy that it's rife with danger.

IMAGEN XX - Representación de los significados de los valores dentro de la escala Fibonacci

2.1 - Ejemplo de User Stories:

# ID: SCRUM-15 - Épica: Módulo de Votación Prioridad: Alta - Estado: Sin asignar Como votante autenticado, quiero emitir mi voto seleccionando una lista de candidatos, para que mi sufragio quede registrado de forma inmutable en la blockchain. Criterios de aceptación: - El votante solo puede emitir un voto por comicio activo; si intenta votar nuevamente, el sistema registra el nuevo voto como válido (voto múltiple, solo cuenta el último). - El voto se registra en el smart contract sin exponer la	SP: 8
---	----------------------------

identidad del votante. El sistema emite un recibo criptográfico con el hash de la transacción al finalizar.		
Pruebas de usuario:		
Escenario	Entrada	Resultado esperado
Voto exitoso	Votante autenticado seleccione lista y confirma	Transacción confirmada en Sepolia; recibo mostrado
Segundo voto	Mismo votante intenta votar nuevamente	Sistema acepta y reemplaza el voto anterior; recibo actualizado
Votante no en padrón	Usuario sin prueba de Merkle válida intenta votar	Sistema rechaza con mensaje de error claro

(!) En caso de corresponder, cada US puede estar acompañada de una imagen de prototipo avalada por el usuario, que represente cómo desea que se vea el producto final.

2.2 - Criterios de aceptación generales:

Más allá de los criterios específicos de cada historia, todas las historias de usuario del proyecto deben cumplir las siguientes condiciones transversales para ser aceptadas:

- El sistema permite ejecutar un proceso electoral completo (desde la configuración hasta la publicación de resultados) en un entorno simulado sin incidentes de integridad.
- Cada votante puede verificar individualmente que su voto fue incluido en el escrutinio mediante el recibo criptográfico emitido por el sistema.
- El Dashboard Público muestra resultados en tiempo real, auditables por cualquier observador externo sin necesidad de autenticación.
- El código fuente se publica bajo licencia open source con documentación técnica suficiente para que otra institución pueda adoptarlo.
- El proyecto se entrega dentro del plazo académico establecido con todos los entregables requeridos por la cátedra.
- La plataforma opera en red Testnet (Sepolia) y el equipo documenta el path de migración a Mainnet.

(!!) Estos criterios son complementarios al Definition of Done del equipo y aplican en paralelo a los criterios específicos de cada historia.

2.3 - Flujo de una historia hasta Testnet:

El desarrollo del producto sigue un flujo iterativo orientado a la calidad y la trazabilidad:

1. **Definición:** Se genera la idea y se formaliza como User Story (US) en Jira con sus criterios de aceptación.
2. **Diseño:** Se elaboran los diagramas técnicos necesarios (secuencia, C4 o modelo de datos) antes de iniciar la codificación.
3. **Desarrollo:** Se implementa la funcionalidad en una rama específica (feature/*).
4. **Validación:** Se abre un Pull Request (PR) para revisión por pares. Si es rechazado por hallazgos, vuelve a desarrollo; si es aprobado, se ejecutan las pruebas automatizadas.
5. **Integración:** Tras la aprobación, se realiza el merge a develop.
6. **Cierre y Deploy:** La funcionalidad se presenta en la Sprint Review y se despliega en la Testnet Sepolia.

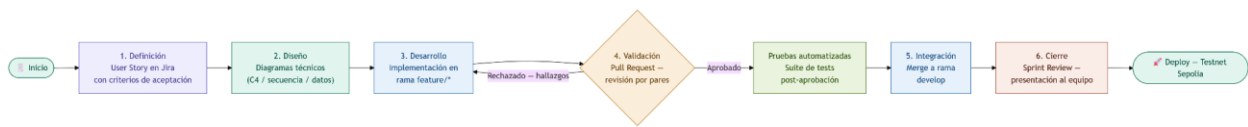


Imagen XX - Flujo de desarrollo del producto

3 - Ceremonias

3.1 - Sprints:

La duración del sprint se estableció de 2 semanas. Las horas ideales dedicadas al mismo variaron en cada uno de los sprints, pero se contó con un rango mínimo y máximo de cada participante en cada iteración.

En los sprints, el equipo implementará las historias de usuario que se comprometió a hacer.

3.2 - Reuniones:

Debido a que la cantidad de actividades a realizar cambia dinámicamente, nuestro equipo de trabajo no quiere ceñirse al concepto de “daily”. Es por eso que vamos a trabajar con el concepto de “Reuniones”. La idea de una “Reunión” es que no dure más de una hora, y que el equipo sea capaz de instanciarla cuando considere necesario. Quizá hay que elaborar un único documento en todo el sprint, entonces con una sola “Reunión” pueden planificarse y asignarse las tareas a realizar. Quizá hay que programar 6 módulos de la aplicación, y ahí el equipo podría instanciar una reunión por día. Trabajar de esta forma permite que el artefacto se adecúe a las actividades a realizar por el equipo, y evita que el equipo tenga que adecuarse a un artefacto que no refleja sus necesidades.

- Documentación de “Reuniones”:

- Se hará uso fundamentalmente de un Google Sheets, o planilla de cálculo, donde se dejará registro de las reuniones instanciadas.
- Los integrantes del equipo se coordinarán vía Whatsapp o Discord para elegir el momento de la reunión y los temas a tratar. Esto se precarga en las columnas “Fecha”, “Sprint N°” (el sprint dentro del cual la reunión tiene lugar), “Motivo”, “Responsable / es”, “Integrantes” y “Tipo de Reunión”.
- Tras la realización de la “Reunión”, se completarán las columnas “Tiempo de reunión” y “¿Qué se hizo?”, lo que en un futuro permitirá consultar los avances realizados o dificultades encontradas a lo largo del proyecto.

3.3 - **Sprint planning:**

Al inicio de cada sprint, el equipo selecciona los ítems del Product Backlog a implementar (que fueron acumulándose en Jira a lo largo del tiempo). Según el objetivo del sprint, se seleccionarán distintas historias de usuario, lo que terminará formando el artefacto “[Sprint backlog](#)”.

3.4 - **Sprint review:**

Al finalizar cada sprint, el equipo llevará a cabo una Sprint Review con el objetivo de realizar una demostración funcional del producto y presentar los incrementos desarrollados durante la iteración. Esta presentación se realizará ante los *sponsors* con fecha a convenir y se enfocará en presentarles los ítems del Product Backlog que fueron completados. En caso de corresponder, se registrarán nuevos ítems o ajustes al Product Backlog de Jira, según las observaciones, errores detectados o posibles mejoras sugeridas por los *sponsors*.

3.5 - **Sprint retrospective:**

Al finalizar cada sprint review, el equipo llevará a cabo una Sprint Retrospective con el objetivo de analizar el proceso de trabajo y detectar oportunidades de mejora para las siguientes iteraciones. A diferencia de la Sprint Review, esta instancia se centra en el equipo y en su forma de trabajo, en lugar del producto desarrollado, y tiene el objetivo de definir acciones concretas de mejora, las cuales podrán traducirse en ajustes en la dinámica de trabajo o en nuevos ítems dentro del Sprint Backlog del siguiente sprint.

- **Documentación de Sprint Retrospectives:** Para documentar esta ceremonia se anotará:
 - **Lo que hicimos bien:** prácticas, decisiones o dinámicas que resultaron efectivas.
 - **Lo que hicimos mal:** dificultades, errores o aspectos que afectaron negativamente el desarrollo del sprint.
 - **Lo que podríamos mejorar:** propuestas concretas de mejora a implementar en futuros sprints.

4 - **Artefactos**

4.1 - **Backlogs y artefactos**

- a. **Backlog del producto:** Este artefacto contiene una lista priorizada de todas las funcionalidades, características y requisitos del producto. En este caso, el Equipo de

Desarrollo en conjunto con el Product Owner fueron los responsables de mantener el backlog actualizado y priorizado. Estas funcionalidades van a estar representadas a través de historias de usuario, por lo que en este artefacto encontraremos una tabla ordenada por prioridad de historias de usuario, con su identificador, nombre, descripción y puntos de historia.

- b. Backlog del sprint: Este artefacto contiene una lista priorizada de todas las funcionalidades, características y requisitos del producto que el equipo se compromete a desarrollar en el sprint en curso. Estas funcionalidades van a estar representadas a través de historias de usuario, por lo que en este artefacto encontraremos una tabla ordenada por prioridad de historias de usuario, con su identificador, nombre, descripción y puntos de historia.

ID	User Story	Descripción	SP

IMAGEN XX - Tabla que será utilizada en la documentación para el product backlog como el sprint backlog.

- c. Burndown chart: Artefacto que permite visualizar diariamente el trabajo restante frente al tiempo disponible, donde el trabajo restante sería las historias de usuario aún no implementadas de forma completa y el tiempo disponible los días del sprint. Aquí veremos un gráfico generado gracias a Jira, que nos permitirá conocer como fue evolucionando la implementación de las historias de usuario en el sprint activo y si finalmente el equipo logró completar los puntos de historia con los que se comprometió.

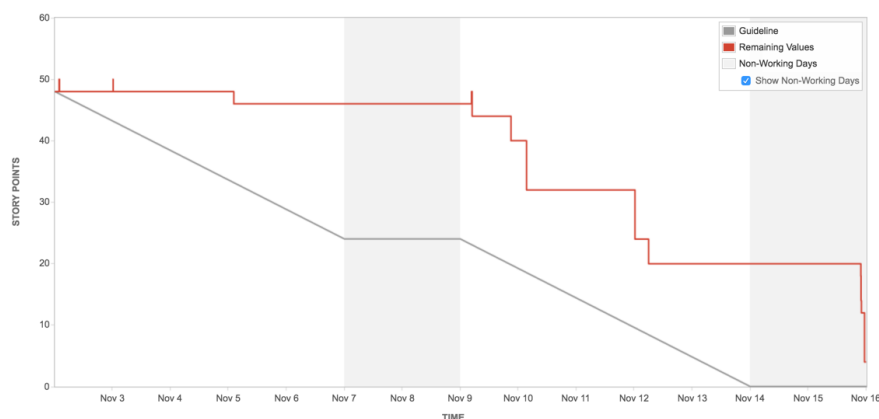


IMAGEN XX - Ejemplo de Burndown chart realizado con Jira.

4.2 - Minutas

- a. Reuniones realizadas: Tras cada reunión (recordemos, una adaptación propia del concepto de 'daily'), vamos a registrar Fecha, Número de sprint, Motivo de reunión, Responsable / es, Integrantes, Tipo de reunión (virtual o presencial), Tiempo de reunión (no podrá ser mayor a 1 hora), ¿Qué se hizo?. Este registro permitirá que las reuniones que se llevan a cabo con el equipo sean verdaderamente significativas y fomenten la realización de trabajo.

- b. Planilla de riesgos: Tras la realización de cada sprint, se registran distintos tipos de riesgos:
- Identificación de riesgos ocurridos** - ID + Descripción + Categoría: Escribimos los riesgos que ya se materializaron en los distintos sprints que realizamos.
 - Riesgos modificados** - ID + Descripción + Modificación + Categoría: Cuando realizamos alguna acción que por ejemplo disminuya la probabilidad de ocurrencia de un riesgo, será anotada en esta sección.
 - Nuevos riesgos identificados** - ID + Descripción + Categoría: Los detectamos, aunque aún no hayan pasado.

4.3 - Métricas:

- a. Cumplimiento de hitos académicos: Al final de cada sprint se realizará un seguimiento binario (cumplido / no cumplido) que realice una correspondencia entre lo solicitado por la planificación de la cátedra y los entregables generados en ese sprint.
- b. Velocidad del sprint: La “velocity” se calcula como los puntos de historia realmente completados en un sprint contra la planificación realizada. Es una muy buena métrica, porque permite ajustar la capacidad estimada del equipo en sprints futuros.
- Jira permite asignar story points a las historias de usuario y errores con los que trabajemos. Abajo vemos una imagen con una historia de usuario de **5 story points**.

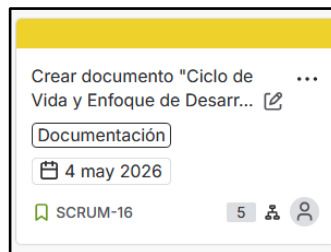


IMAGEN XX - Historia de usuario (de carácter ilustrativo) con 5 SP.

- En la imagen XX que se muestra a continuación podemos ver cómo los story points se contabilizan a nivel de sprint. Arriba a la derecha pueden verse los números [5, 2, 5]. Los mismos indican la cantidad de story points [Pendientes, En curso, Hechos], lo que nos permite obtener los puntos de historia planificados (sumatoria de todos los SP para ese sprint) y los puntos de historia completados (SP en “Hechos”), lo que permitirá el cálculo de la velocity del equipo.



IMAGEN XX - Story points en sprint (arriba a la derecha)

5 - Estructura del equipo y Matriz RASCI

Según el PMBOK 7ma edición, la definición de roles y responsabilidades forma parte del dominio de rendimiento del equipo (*Team Performance Domain*), y su correcta formalización es condición necesaria para que un equipo opere con eficiencia. Con este fundamento, el equipo Five Stack adoptó la herramienta RASCI (Responsible, Accountable, Support, Consulted, Informed) para eliminar ambigüedades sobre quién ejecuta, quién decide y quién debe ser informado ante cada entregable.

El equipo está integrado por cinco estudiantes que asumen roles diferenciados más allá del desarrollo técnico. Todos los roles son fijos durante el ciclo académico hasta finales de julio: el rol de Director del Proyecto recae en Ignacio Mosconi, quien centraliza la gestión, la comunicación con la cátedra y el control del cronograma, mientras que los restantes roles de liderazgo técnico se distribuyen de forma estable entre los demás integrantes. A partir de agosto, Bruno Lucarelli debido a compromisos académicos de carácter inevitable; no obstante, esta situación no comprometerá el avance del proyecto, dado que uno de los integrantes absorberá sus responsabilidades dentro de su rol actual.

Rol	Criterio de asignación, primera etapa	Criterio de asignación, segunda etapa
Director del Proyecto (PM)	Ignacio Mosconi	Ignacio Mosconi
Líder de Arquitectura / Blockchain	Gastón Magni	Gastón Magni
Líder de Backend / Seguridad	Valentino Terreno	Valentino Terreno
Líder de Frontend / UX	Bruno Lucarelli	Alejo Liendo
Líder de QA / Documentación	Alejo Liendo	Alejo Liendo

5.1 - Asignación de roles por fase - Matriz RASCI:

La matriz RASCI asigna a cada actividad del proyecto uno de cinco roles posibles por integrante: **R** (*Responsible*) es quien ejecuta la tarea; **A** (*Accountable*) es el único responsable final del resultado y quién responde ante los interesados; **S** (*Support*) colabora activamente en la ejecución; **C** (*Consulted*) aporta criterio experto antes de tomar decisiones; e **I** (*Informed*) recibe comunicación sobre el avance sin intervenir directamente. Una actividad puede tener múltiples R y S, pero siempre un único A.

A fin de aplicar esta herramienta de forma concreta y adaptada a la dinámica del proyecto, en cada sprint se elaborará una matriz RASCI específica, reflejando las responsabilidades reales de cada integrante según las tareas planificadas para ese período. Esto permite que la asignación de roles no sea estática ni genérica, sino que evolucione sprint a sprint en función de las actividades concretas, el contexto del equipo y la etapa del ciclo de vida del proyecto. De este

modo, cada integrante tiene visibilidad clara sobre su nivel de participación esperado en cada tarea, facilitando la rendición de cuentas y la coordinación interna.

5.2 - **Requerimientos:**

Toda necesidad funcional o técnica identificada por el equipo o los interesados se documenta como una Historia de Usuario en Jira. Cada historia incluye criterios de aceptación explícitos, estimación en puntos de historia y prioridad dentro del backlog.

5.3 - **Roles en Scrum:**

- **Roles del equipo:** Si bien la matriz RASCI nos permite definir los roles para las distintas tareas a realizar dentro del sprint, es oportuno destacar que dentro de cada sprint los miembros del equipo pueden tomar dos roles distintivos:
 - **Scrum Master:** El encargado de llevar los registros de las reuniones, actualizar los backlogs, etc. Normalmente Ignacio Mosconi va a ocupar este rol, pero el mismo puede ser rotado entre sprints.
 - **Equipo de Desarrollo:** Resto de los integrantes.
- **Roles de los interesados:** Los interesados en el proyecto (Ing. Villafañe e Ing. Cassani) reciben el nombre de “sponsors”, que son nuestra versión del Product Owner de Scrum. El equipo de trabajo deberá constatar las entregas realizadas con los *sponsors* para saber que no hay que realizar más modificaciones sobre ellas.

6 - **Flujo de Scrum simplificado**

Ahora que conocemos varios conceptos, podemos hablar de cómo va a fluir un día normal de un equipo de Scrum.

Durante el **sprint planning** el equipo organizará la forma en la que va a trabajar en cada sprint. Aquí se seleccionarán, desde un **product backlog** que en esta misma instancia se aprovecha a actualizar respecto a lo avanzado en el sprint anterior, las **user story / épicas** a las que el equipo se compromete a realizar en este sprint, formando así el **sprint backlog**.

Una vez el sprint backlog esté completo, se completará la **matriz RASCI**, lo que permitirá asignar responsables a los distintos miembros del equipo. Tras realizar esto, se dará inicio al **sprint**.

Durante un sprint, el equipo puede instanciar distintas **reuniones**: si hay mucho trabajo por hacer, puede tomarse lo dictado por scrum e instanciar una reunión por día (generando así una daily), pero si el trabajo es relativamente poco, el equipo puede llegar a hacer dos o inclusive una reunión por semana.

A medida que se vaya realizando el sprint, el equipo podrá encontrarse con **riesgos**, los cuales serán documentados con las clasificaciones mencionadas anteriormente. El equipo también irá actualizando sus avances en **Jira**, lo que permitirá realizar un seguimiento de los *pendientes* aún sin resolver.

Cuando termine el tiempo del sprint (recordemos, serán 2 semanas por sprint) el equipo podrá realizar una **sprint review** (donde los entregables se constatan con los sponsors) y la **sprint retrospective** (donde equipo de trabajo comenta su experiencia en el sprint y qué aspectos pueden mejorarse a futuro).

Lo último que le queda al equipo será realizar el **burndown chart**, calcular la **velocidad del sprint** y validar el **cumplimiento de hitos académicos**, dando así lugar a la finalización del sprint.

Scrum Process

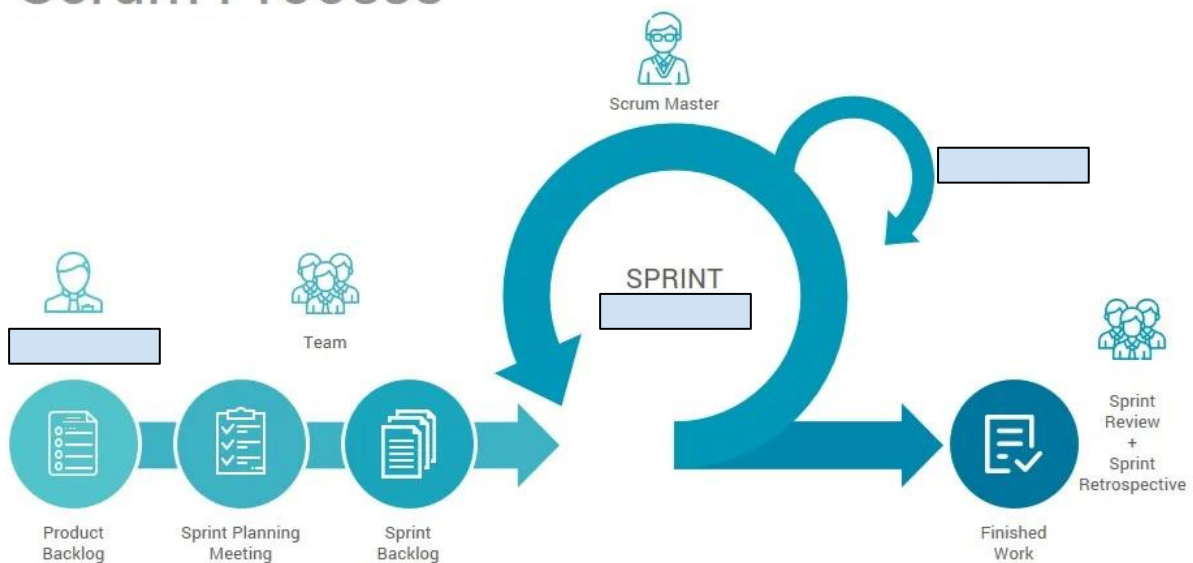


Imagen XX - Ciclo de Scrum

7 - El rol de Jira en nuestros Sprints

Como se mencionó en el punto anterior, el equipo utilizará Jira para realizar un seguimiento de los *quehaceres* de un sprint.

Nos pareció importante tener una sección para destacar el uso de esta herramienta para la gestión el proyecto, lo cual puede visualizarse a través de las siguientes imágenes:



Imagen XX - Gestión de product backlog y sprint "Documentación" en Jira

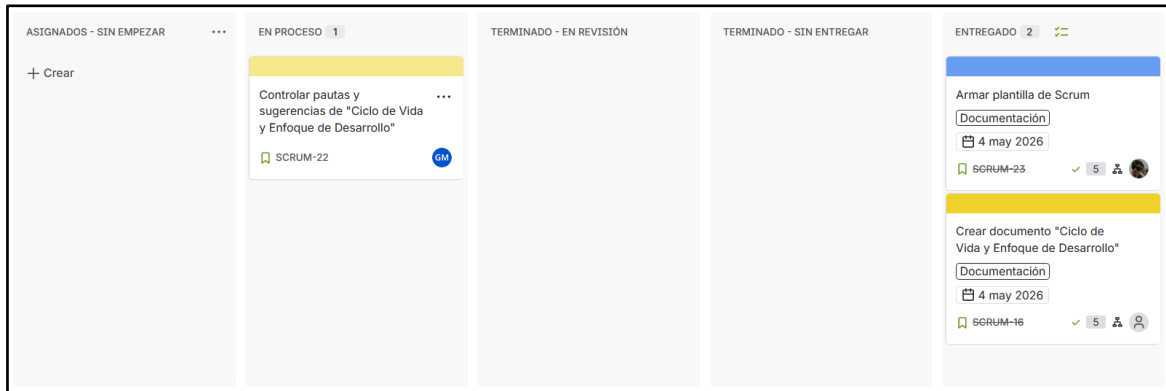


Imagen XX - Tablero Kanban utilizado para gestionar el sprint “Documentación” mostrado con anterioridad. El mismo fue creado para mostrar el funcionamiento de Jira.

La flexibilidad de Jira nos deja organizar nuestro Product Backlog como queramos. En nuestro caso definimos **cuatro niveles de abstracción** dentro de Jira:

1. **Historias de usuario**: Las historias realizan un seguimiento de la funcionalidad o las características expresadas como metas del usuario. Se expresan con el formato “*Como [rol], quiero [acción], para [beneficio]*” y con **criterios de aceptación** claros. Son el eje central de nuestro tablero: **todo el trabajo que hagamos deberá ser en base a una historia de usuario**.
 - a. Una historia de usuario puede estar formada por **subtareas** ([ver imagen XX](#)).
2. **Errores**: Los errores realizan un seguimiento de distintos problemas, ya que por ejemplo pueden instanciarse para resolver problemas de historias de usuario ya declaradas como entregadas.
3. **Épicas**: Las Épicas son colecciones de Errores e Historias de usuario, por lo que pueden definirse como agrupaciones de funcionalidad de alto nivel (ej. Módulo de autenticación, Módulo de Votación, Dashboard Público, Sistema de Auth, etc.).

Las siguientes imágenes muestran la organización del flujo de trabajo en el Product Backlog:

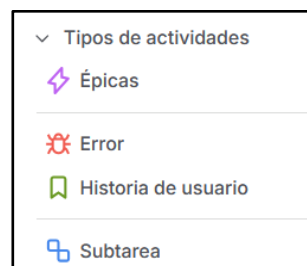


Imagen XX - Niveles de abstracción que definimos para trabajar en nuestro espacio Jira.

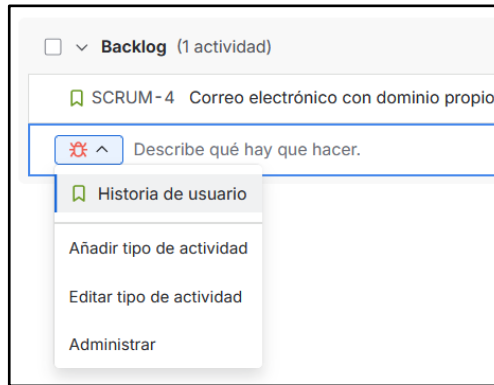


Imagen XX - Al momento de crear una nueva entrada al product backlog, podemos seleccionar si será un **error** (actualmente seleccionado) o una **historia de usuario**.

Subtareas					...	+
0 % completado						
Resumen	Prioridad	Story point estimate	Estado	Persona asignada		
Desarrollar informe	🔴 Highest	3	EN PROCESO	BL Bruno L...		
Revisar informe	🔴 High	2	ASIGNADOS - SIN EMPEZAR	AL Alejo Lie...		
Enviar correo con informe	🟡 Medium	0	ASIGNADOS - SIN EMPEZAR	Ignacio ...		
Cerrar sprint	🟡 Medium	0	ASIGNADOS - SIN EMPEZAR	Ignacio ...		

Imagen XX - La única forma de crear subtareas es dentro de historias de usuario ya creadas.

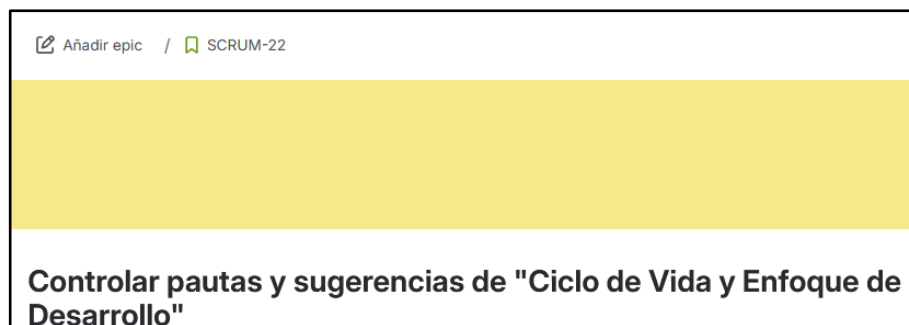


Imagen XX - Como se visualiza arriba a la izquierda de la imagen, podemos añadir una Épica a una, Error o Historia de Usuario ya creados (en este caso se muestra una historia de usuario).

ENFOQUE DE DESARROLLO DEL PRODUCTO - COMPONENTE PREDICTIVA

Introducción

Todo lo que definimos de nuestro Scrum Simplificado puede considerarse la **componente adaptativa** del proyecto, ya que justamente nos permite realizar tareas de forma ágil y reactiva al ambiente.

Nuestro proyecto tendrá además una **componente predictiva**, es decir que utilizará lógica definida de antemano. No, no es que vamos a trabajar en cascada, es que hay ciertas cosas que pueden aplicarse a todos los procesos de dirección de proyectos, como la [gestión de configuración](#).

Herramientas del proceso

Herramienta	Propósito	Modo de uso
Git / GitHub	Control de versiones y colaboración	Branching por feature; PRs con revisión de pares
Jira (Atlassian)	Gestión visual del backlog y sprints	Columnas: Asignados / En proceso / En revisión / Terminado - Sin entregar / Entregado
Google Drive / Docs	Documentación del proyecto	Versionado colaborativo de entregables académicos
Hardhat / Ganache	Entorno local de desarrollo blockchain	Pruebas de smart contracts sin conexión a Testnet
Infura / Alchemy (free tier)	Nodo RPC en Sepolia Testnet	Despliegue y pruebas en red pública de prueba
WhatsApp / Discord / Google Meet	Comunicación del equipo	Reuniones y coordinación

Plan de Gestión de la Configuración

La Gestión de la Configuración es el proceso responsable de administrar los entregables tanto del proyecto como del producto a lo largo de su desarrollo, permitiendo un control sistemático de los cambios y sus versiones. Este proceso abarca la identificación, documentación, evaluación y control de cualquier modificación sobre las líneas base del proyecto (incluyendo alcance, cronograma y producto). La estrategia del equipo establece dos enfoques paralelos: una Gestión de la Configuración del Proyecto, destinada a controlar las modificaciones de todos los documentos y entregables de gestión, y una Gestión de la Configuración del Producto, enfocada netamente en el desarrollo y en la administración del repositorio de implementación.

Gestión de la configuración (SCM) del proyecto

Para la gestión de la configuración de la documentación, se utilizará Google Drive como repositorio principal, cuyo acceso estará restringido exclusivamente al equipo de trabajo. Adicionalmente, se establecerá un directorio compartido con el equipo docente de la cátedra, donde se alojarán todas las versiones de los entregables para su posterior revisión y control.

Con el objetivo de mantener un formato estandarizado en toda la documentación, se empleará principalmente la suite de Google Workspace: Google Docs para los documentos de texto, Google Sheets para el registro de minutas y Google Drawings para esquemas básicos. Para el diseño de presentaciones se utilizará Canva. Asimismo, la elaboración de diagramas específicos y modelos más complejos se realizará mediante software externo (como excalidraw, draw.io / Diagrams.net, etc.), asegurando que los archivos o exportaciones resultantes también se centralicen en el repositorio principal de Google Drive.

Versionado documental

Todos los documentos del proyecto siguen el esquema de versionado [a].[b].[c] definido en el Project Charter:

- [a]: número de versión principal, incrementado con cada devolución de la cátedra y aplicación de correcciones.
- [b]: incremento por desarrollo de nuevos títulos o secciones del documento.
- [c]: revisión interna del equipo sobre un incremento [b] previo, sin intervención de la cátedra.

Los documentos se almacenan en Google Drive bajo una estructura de carpetas por entregables. La carpeta de cada documento contiene la versión más reciente y un historial de versiones anteriores. El historial de revisiones dentro de cada documento detalla la fecha, el autor y la descripción del cambio para cada versión.

Procedimiento formal de cambios

Durante la edición de cada documento se realizan comentarios y sugerencias. Si se presentan dudas o cambios, se debe notificar mediante los canales de comunicación asincrónicos preestablecidos. Antes de realizar una entrega formal, el equipo debe revisar, corregir y verificar los documentos en reuniones presenciales o mediante alguna plataforma virtual de preferencia. Cada entrega de uno o más documentos se realiza subiendo dicho entregable a la carpeta correspondiente dentro del repositorio compartido con los docentes de la cátedra y se envía un correo formal a la cátedra, para notificar que ya se encuentra la documentación disponible para su revisión.

Roles e integrantes del comité

El comité de gestión de cambios será conformado por todos los miembros del equipo. Cada integrante formará parte de este con el rol correspondiente, a excepción del líder que será el encargado de organizar y mediar la reunión de revisión de documentación.

Procesos de auditoría

Todos los documentos entregados deberán pasar por una revisión llevada a cabo por el equipo de desarrollo, controlando que todo el contenido requerido esté presente y detallado en el documento.

Gestión de la configuración del producto

Sistema de control de versiones (Software)

Todo el código fuente del proyecto se gestiona en un repositorio Git alojado en GitHub, bajo la organización del equipo Five Stack. La estrategia de branching adoptada es **GitHub Flow** simplificado:

Rama	Propósito
master	Rama principal de producción. Solo recibe merges desde develop una vez que el incremento supera todas las pruebas y es aprobado por el equipo.
develop	Rama de integración continua. Todos los features se integran aquí antes de ser promovidos a master.
feature/[nombre]	Ramas de feature por historia de usuario o tarea técnica. Nomenclatura: feature/modulo-descripcion-breve (ej. feature/auth-merkle-proof).
fix/[nombre]	Corrección de bugs identificados en develop o main. Nomenclatura análoga a las ramas de feature.
docs/[nombre]	Actualizaciones de documentación técnica (README, guías de despliegue, etc.).

Las reglas de integración son:

- Ningún integrante puede hacer merge directo a master sin Pull Request (PR)
- Todo PR requiere la revisión y aprobación de al menos un integrante distinto al autor
- El PR debe pasar las pruebas automatizadas antes de poder ser mergeado
- Los mensajes de commit siguen la convención establecida en el siguiente título (*convención para commits*).

Convención para Commits

Los cambios se documentarán utilizando los commits, para que el historial de desarrollo del producto sea legible, fácilmente escaneable y entendible por cualquier persona que participe en el proyecto. Para la creación de los commits, el equipo se comprometió a cumplir con las siguientes pautas:

Todos los cambios comprometidos en un commit deben ser lo más atómicos e independientes posibles. Para identificar el cambio se deberá utilizar prefijos para darle más significados a los cambios realizados de la siguiente manera:

<tipo-de-commit>[scope]: <descripción>

Los prefijos utilizados serán los siguientes:

- feat: Una nueva característica para el usuario.
- fix: Arregla un error que afecta al usuario.
- perf: Cambios que mejoran el rendimiento del sitio.
- build: Cambios en el sistema, tareas de despliegue o instalación.
- ci: Cambios en la integración continua.
- docs: Cambios en la documentación.
- refactor: Refactorización del código como cambios de nombre de variables o funciones.
- test: Añade pruebas o refactoriza existentes.

Cuando se tenga que especificar el scope, se tomará en cuenta la funcionalidad en la que se está trabajando principalmente en ese commit. Por ejemplo:

feat(backend): add filter for customer

Para describir el cambio se deberá utilizar el verbo imperativo, de forma que cada commit se interprete como una instrucción para cambiar el estado del proyecto. A continuación, se enuncian algunos verbos sugeridos por el equipo:

- Add: Se agrega un nuevo archivo o funcionalidad
- Change: Se modifica un archivo
- Update: Se actualiza un archivo (Cambios pocos significantes)
- Upgrade: Se mejora el archivo, pero no se cambia su funcionalidad
- Fix: Se arregla un problema de un archivo
- Remove: Se elimina un archivo

Gestión de cambios al alcance

Cualquier cambio al alcance del producto o del proyecto, una vez que el Scope Statement ha sido aprobado, sigue el siguiente proceso formal de control de cambios:

Paso	Actividad	Descripción
1	Identificación	Cualquier integrante o stakeholder puede identificar un cambio necesario.
2	Documentación	El cambio se registra en el Issue Tracker de GitHub con la etiqueta "change-request", detallando: descripción, justificación, impacto estimado en alcance/tiempo/costo y alternativas.
3	Evaluación	El Director del Proyecto analiza el impacto y el equipo evalúa la viabilidad técnica en la siguiente reunión de sprint.
4	Decisión	El equipo decide por consenso. Si el cambio afecta entregables académicos, se eleva a consideración de la cátedra.

Paso	Actividad	Descripción
5	Implementación	Si se aprueba: se actualiza el Scope Statement, la EDT y el backlog. El issue queda vinculado a los commits de implementación.
6	Comunicación	El Director del Proyecto notifica a todos los interesados relevantes sobre el cambio aprobado y su impacto.

Gestión de entornos

El proyecto mantiene tres entornos claramente separados para garantizar la estabilidad y la trazabilidad de los despliegues:

Entorno	Infraestructura	Rama asociada	Propósito
Local (DEV)	Hardhat / Ganache en máquina del desarrollador	feature/* / develop	Desarrollo y pruebas unitarias sin costo de gas
Testnet (QA)	Nodo RPC Sepolia vía Infura/Alchemy free tier	develop / main	Pruebas de integración y demo piloto en red pública
Mainnet (PROD)	Red Ethereum pública - FUERA DEL ALCANCE	N/A	No aplica durante el ciclo académico

Gestión de artefactos de release

Al finalizar cada sprint y al cerrar cada hito del proyecto, se genera un release en GitHub con las siguientes características:

- Tag de versión semántica (vMAJOR.MINOR.PATCH)
- Release notes automatizadas a partir de los títulos de los PRs mergeados
- Artefactos de despliegue (ABI de los contratos, imagen Docker del backend si aplica)
- Referencia a la dirección del smart contract desplegado en Testnet para auditoría externa.

La trazabilidad completa entre historias de usuario, commits, PRs y releases es un requisito no negociable del proceso de desarrollo de VOTAR, alineado con los principios de transparencia y auditabilidad que definen la plataforma como producto.

Roles e integrantes del comité

El comité de gestión de cambios será conformado por todos los miembros del equipo. Cada integrante formará parte de este con el rol correspondiente, a excepción del líder que será el encargado de organizar y mediar la reunión de revisión de documentación.

Procesos de auditoría

Durante el sprint y al finalizar el mismo se verificará que la rama de desarrollo sea correcta. Si la rama es correcta y los entregables son aceptados, entonces toda la rama podrá unificarse con master. Este merge debe pasar los estándares definidos anteriormente.

BIBLIOGRAFÍA

→ Gestión del proyecto basada en PMBOK:

- Project Management Institute. (2021). A guide to the project management body of knowledge (PMBOK guide) (7th ed.).

→ Diagramas de secuencia:

- El Lenguaje Unificado de Modelado: Guía del Usuario - Booch, G., Rumbaugh, J., & Jacobson, I. (2006). El lenguaje unificado de modelado: Guía del usuario (2.ª ed., cap. 19: Diagramas de interacción). Pearson Educación.
- El Lenguaje Unificado de Modelado: Manual de Referencia - Rumbaugh, J., Jacobson, I., & Booch, G. (2006). El lenguaje unificado de modelado: Manual de referencia (cap. 8: La vista de interacción). Pearson Educación.

→ Scrum y enfoques ágiles:

- Ries, E. (2011). *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. Crown Business.
- VII, P. (2016). *Minimum Viable Product: 21 Tips for Getting an MVP, Early Learning and Return on Investment for Your Product*. Pashun Consulting Ltd.
- Steve Mc Connell - Rapid Development Redmond, Wa.: Microsoft Press,
- Cohn, Mike – Agile Estimation and Planning – Editorial Prentice Hall 2006
- Cohn, Mike - User Stories Applied – Editorial Addison Wesley 2004.

→ C4:

- Software Architecture for Developers - Brown, S. (2015). Software architecture for developers: A practical guide to building and documenting software architectures. Leanpub.